Week 5 | In-class challenge

Proposed solution prepared from the prompt; no completed team response was supplied. [1, 2]

Need	Technique	Reason
a. Logbook shorthand	CPT, if needed	CPT learns domain phrasing; first test a glossary plus a pretrained model.
b. Current APU MEL + citation	RAG	Retrieve the approved revision filtered by fleet and aircraft applicability, and cite it.
c. Route defect to ATA chapter	CFT	A supervised classification head maps the write-up to a fixed ATA label set.
d. Standard shift summary	Prompting, then IFT	Try a template with examples; use IFT only if format or coverage remains inconsistent.
e. Tire replacements last quarter	NL-to-SQL	Query dated fleet records with validated definitions; document retrieval cannot reliably count events.
f. No sign-off; uncertainty	DPO + guardrails	Expert preferences teach deferral; explicit application rules block sign-off claims and unsupported answers.

2. Sizing the project

Assume **decimal GB** and the lecture's averages: $10 \times 10^9 / 4$ = **2.5B tokens**. Scans, images, tables, duplicates and boilerplate reduce usable text; measure tokens after OCR, cleaning and deduplication. A 3B model wants roughly 20 x 3B = **60B tokens** under Chinchilla: this corpus is **24x too small** (4.17% of the rule-of-thumb target). Scratch training is technically possible, but is a poor data-efficient choice; repeating the same corpus does not add that much distinct information.

One 8B CPT epoch: $C = 6 \times 8 \times 10^9 \times 2.5 \times 10^9$ = **1.2×10^{20} FLOPs**. At 4×10^{14} FLOP/s: 300,000 GPU-seconds = **83.3 H100 GPU-hours**. This is aggregate compute, before overhead or extra data; mixed-precision Adam also needs about **128 GB + activations**, so sharding may be needed. Real cost: clean, permitted data, expert labels/evaluation, integration and revision control as well as compute.

3. Evaluate before training

Maintenance engineers, technicians and dispatch SMEs write a held-out golden set with source revisions, correct answers and rubric scores. Include shorthand, aircraft applicability, updated/superseded procedures, unanswerable questions, ATA routing, summaries and SQL counts. Split by time and aircraft/work-order groups; keep related records and paraphrases out of training. Baselines: **(1)** general model + prompt/glossary, **(2)** that model + hybrid RAG, **(3)** small open model + the same RAG. Score evidence-supported correctness, citation validity, harmful errors, abstention, routing macro-F1, summary fidelity, exact counts and latency. Stop without training if an untrained baseline meets agreed quality, reliability and deployment targets and training offers no material held-out gain.

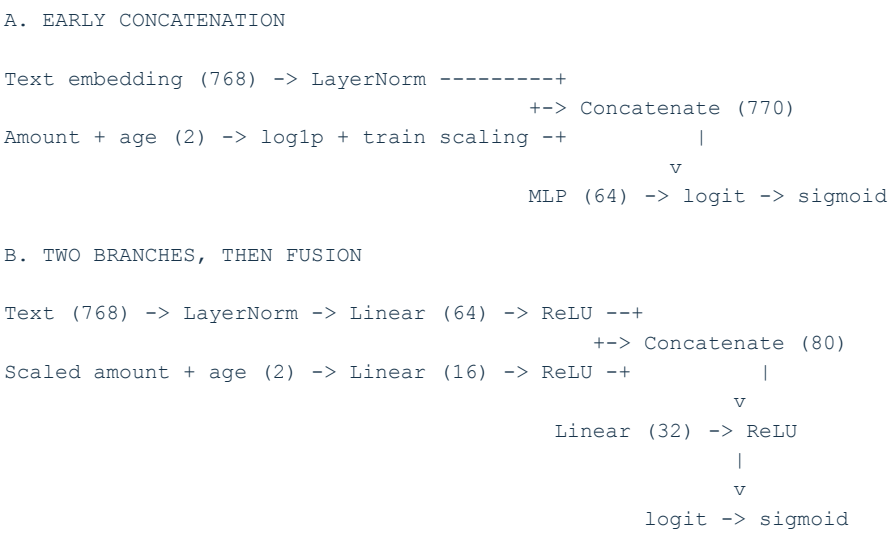
4. Risk and recommendation

Largest weights risk: confidently recalling a superseded procedure without traceable revision evidence. Keep authoritative procedures in versioned retrieval; validate applicability/citations and abstain on gaps. **Data risk:** OCR can corrupt part numbers, units or limits; review critical tables against approved originals. **Recommendation to the VP:** start with evaluated hybrid RAG and database queries, add task-specific CFT/IFT where needed, and use CPT only for a demonstrated language gap.

Question 1 | Combine the features

Recommendation: a small two-branch classifier, tested against a strong 770-dimensional baseline.

Let **e** be the supplied 768-dimensional text embedding and **s** contain amount and account age. Assume all features are available at transaction decision time and the target is a binary fraud label. Freeze the text encoder initially so the comparison tests the fusion design.



A. Concatenate first

Yes: concatenate [e; s] to obtain **770 features**, then train logistic regression or an MLP using fraud labels. A learned layer has a separate coefficient for each input; an MLP can also learn interactions between text and numbers. This is a reasonable, inexpensive baseline. A linear model alone cannot learn nonlinear interactions without explicit feature engineering.

B. Process separately, then fuse

Use LayerNorm(e) -> Linear(768, 64) -> ReLU for text, and scaled s -> Linear(2, 16) -> ReLU for structured features. Concatenate the resulting **80-dimensional representation**, then use Linear(80, 32) -> ReLU -> Linear(32, 1). Train both branches and the fusion head jointly with binary cross-entropy on logits; apply sigmoid for inference. Separate branches permit modality-specific preprocessing, capacity and regularization, while the fusion MLP learns cross-modal interactions.

Transform the two variables before either architecture

For nonnegative, right-skewed amount and age, try **log1p**, then standardize each feature using training-set means and standard deviations. Compare this with ordinary or robust scaling on validation data; log transforms are a defensible starting choice, not a requirement. If refunds can be negative, use a signed-log transform or a separate refund flag. Validate units and ranges, impute missing values using training-set statistics, and consider missingness flags. Two extra flags would make the simple input **772** rather than 770 dimensions.

Keep preprocessing fixed at inference. Normalize the text representation separately if its scale warrants it; LayerNorm in the accompanying example is one option. Standardizing numbers controls scale, but does not force equal influence across modalities.

Sources: homework Q1 [3]; trainable projections and concatenation in the supplied attention notebooks [4, 5].

Question 1 | Influence and validation

Feature count is not the same thing as predictive importance.

For a linear fusion model, write the logit as:

```
z = w_text^T e + w_amount s_amount + w_age s_age + b
```

The two numeric coefficients can be large while most text coefficients are small or zero. For example, set all 768 text coefficients to zero and use **z = 3s_amount - 2s_age**. Holding age and text fixed, changing standardized amount from 0 to 1 changes sigmoid(z) from **0.500 to 0.953**. The two structured variables determine the decision despite having fewer dimensions. This is a constructive example, not a claim about real fraud data.

Dimension imbalance can still affect optimization: a large text block may have a larger total norm, many correlated features, or more opportunities to fit noise. Scaling, training signal, regularization, initialization and data quality matter. Expanding two numbers into 16 hidden units does not create new information or guarantee their importance.

Architecture	Benefit	Limitation
Early concatenation	Simple; few design choices; useful low-variance baseline.	One shared stack must handle both modalities; poorly scaled inputs can impair training.
Two branches + fusion	Separate transforms and capacity; joint nonlinear interactions.	More tuning; bottlenecks may discard signal; extra capacity can overfit.

Recommendation and decision rule

I recommend **two branches with a modest fusion head** because the features differ in scale and structure, and amount/age may interact with textual patterns. Keep early concatenation as the control and select it instead if held-out results are equivalent. Match tuning effort and regularization; do not infer superiority from the diagram.

Use chronological validation and prevent account or linked-transaction leakage as appropriate to deployment. Fit all preprocessing on training data only. Compare text-only, structured-only, early fusion and two-branch models. For imbalanced fraud, assess precision-recall AUC, recall at the permitted false-positive rate, calibration and cost-based thresholds. Use feature ablations or permutation tests, interpreted carefully when inputs are correlated, to check incremental contribution.

Accompanying Python and references

Week5_Solutions.py reproduces the challenge arithmetic and storage estimates, implements both fusion models, checks finite gradients, demonstrates the numerical counterexample above, and trains a tiny attention module without changing its size. These are synthetic checks; no fraud dataset or real maintenance corpus was supplied. Run the file for printed results; use **--pdf-only** to regenerate this document from **Week5_Homework_Solutions.md**. Dependencies: PyTorch for demonstrations; ReportLab and markdown-it-py for PDF generation.

[1] Week5_Challenge_Airline_Domain_Model.html, Tasks 1-4.
[2] Week5_Lecture_Note_Domain_Models.html, sections 2 and 4-8.
[3] Week5_Homework.html, Questions 1-3 and submission requirements.
[4] Karpathy_GPT_Teaching_Edition.ipynb, sections 4-8; raw cells 20-22 and 48-50.
[5] Self_Attention_Sebastian_CH3.ipynb, sections 3.4-3.6; raw cells 24, 68, 87, 90 and 128.
All references are the supplied course files. Hardware/storage figures are estimates under stated assumptions, not device benchmarks.

Question 2 | What learning changes

A new skill usually changes parameter values; it does not require a larger architecture. [2, 4, 5]

Parameters are learned numbers

A model parameter is a learned scalar in a weight matrix, bias vector, embedding table or normalization scale/offset. Parameters determine how inputs become predictions. During training, a loss measures prediction error, backpropagation computes gradients, and an optimizer updates the selected trainable parameters. A simple update is:

```
theta_next = theta_now - learning_rate * gradient(loss)
```

The number and shapes of the tensors can remain fixed while their entries change. Learning rate and layer count are hyperparameters; hidden activations and attention probabilities are computed from an input, rather than being persistent learned parameters. Optimizer state and gradients consume training memory but are not extra inference-model parameters.

Same architecture, different behavior

In attention, $\mathbf{Q = XW_Q}$, $\mathbf{K = XW_K}$, $\mathbf{V = XW_V}$. Updating the existing projection matrices changes which context is emphasized and which features are passed forward, even when all matrix shapes stay unchanged. The Raschka notebook makes these projections trainable in its SelfAttention classes; its earlier frozen matrices are an illustration, not a training requirement. The Karpathy notebook's loop uses loss.backward() and optimizer.step() on an already constructed model. That loop changes values, not the number of layers.

Technique	Learning signal and resulting change	Must model size grow?
CPT	Raw domain text; next-token loss updates existing language-model weights for jargon and phrasing.	No. Same vocabulary, blocks and output head can be retained.
IFT	Instruction-response pairs; response-token loss updates existing weights for task behavior and format.	No. Same model can learn a new response pattern.
DPO	Preferred/rejected answer pairs; updates the policy's existing weights toward expert preferences.	No. A reference policy can be used during training without enlarging the deployed policy.
CFT	Labeled examples; a K-class head learns scores from the final hidden representation.	A new head introduces parameters, but may replace a much larger vocabulary head.

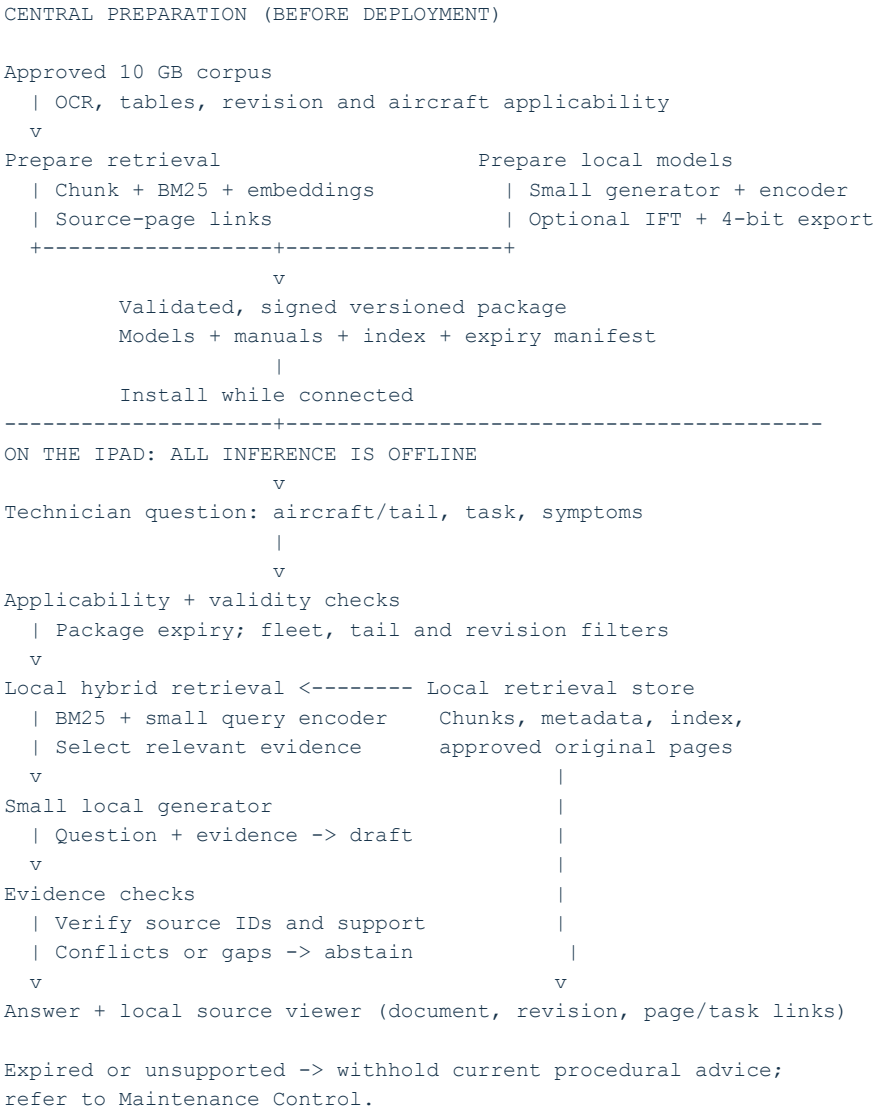
A concrete case that introduces parameters

For ATA routing, attach a classification head **W of shape d x K**, with an optional K-element bias: **dK + K new learned values**. Retaining the language-model head and adding this head increases the total count by that amount. Replacing the vocabulary head introduces a new parameter set but can **reduce** the total, since K is far smaller than the vocabulary size; the exact net change depends on whether output and input embeddings share weights. The backbone may be frozen or updated. The lecture's CFT example is this architectural change.

Thus, adaptation can mean changing existing values, adding a component, or replacing one. RAG is a useful contrast: retrieving passages changes the input context and local data store without updating generator weights. The Python example verifies that an **18-parameter attention module** changes its outputs after training while retaining all tensor shapes; its toy loss is only an illustration of this mechanism.

Question 3 | Offline architecture

One diagram: central preparation is allowed; every inference step runs on the iPad. [2, 3]



Weights supply language and task behavior. The installed retrieval package supplies the authoritative, versioned maintenance evidence. The following page addresses the four requested design questions.

Question 3 | Design justification

One-page justification - four requested decisions. [2, 3]

1. On the device

Run an instruction-tuned **small language model, initially about 1B parameters**, plus a compact text embedding encoder and local BM25 search. Quantize generator weights to 4-bit, use an iPad-compatible accelerated runtime, generate one answer at a time, and cap context length and output length to bound KV-cache and activation memory. A 1B model has roughly **0.5 GB of packed weights**; 3B would be about 1.5 GB, before quantization metadata, cache, encoder and runtime overhead. Move toward 3B only if measured accuracy, peak RAM, latency and thermal behavior pass on the actual target iPad. A cached glossary and direct source viewer support abbreviation handling and fallback.

2. Where knowledge lives

Keep the 10 GB corpus **primarily in local storage**: approved originals, extracted procedure chunks, revision/applicability metadata, and a hybrid retrieval index. Weights provide general language ability and optional task training, rather than serving as the source of current procedures. Disk capacity and RAM are different constraints: keep documents and indexes on disk and load bounded search results. As an upper text scenario, 2.5B tokens in 500-token windows with 100-token overlap yield about 6.25M chunks; 384-dimensional FP16 vectors alone need **4.8 GB**, before search-index overhead. Measure extraction yield, compress or memory-map the index, and reserve storage for originals plus updates. Do not assume the entire store fits in RAM.

3. Techniques and question flow

Use **local RAG** for current, citable facts; keyword search preserves exact part numbers and ATA codes, while a local encoder retrieves semantic matches. Filter by aircraft configuration, task and approved revision, then feed a few complete, relevant procedure passages to the small generator. The answer must link to local document pages and task IDs. Start with prompting; add **IFT** centrally on expert-verified, retrieval-grounded examples if the small model needs help with citations, format or abstention. **Quantization** reduces inference memory. Reserve **CPT** for demonstrated shorthand errors after glossary/retrieval improvements; it is not the update channel for procedures. **CFT** is unnecessary for this initial question-answering workflow; a routing head is optional if later evaluation justifies one.

4. Reliability as procedures change

Centrally ingest approved revisions, verify OCR/tables and aircraft applicability, rebuild affected chunks/index entries, and remove superseded entries from active search. Ship a signed package containing matching documents, index, models and a version/expiry manifest; install it atomically after integrity checks when connected, with a validated rollback copy. Test retrieval recall, citation support, revised facts, conflicting sources and unanswerable cases with maintenance SMEs after every package or model change. At inference, validate cited IDs and evidence support; use direct excerpts or abstain when support is weak. Display revision dates and offline freshness. An offline device cannot discover unseen revisions: after package expiry or a known applicability conflict, withhold current procedural advice and direct the technician to Maintenance Control. The assistant supplies cited support; it cannot authorize dispatch or sign off maintenance.